

A curved-element unstructured discontinuous Galerkin method on GPUs for the Euler equations

M. Siebenborn · V. Schulz · S. Schmidt

Received: 26 September 2012 / Accepted: 13 May 2013 / Published online: 17 October 2013
 © Springer-Verlag Berlin Heidelberg 2013

Abstract In this work we consider Runge–Kutta discontinuous Galerkin methods for the solution of hyperbolic equations enabling high order discretization in space and time. We aim at an efficient implementation of DG for Euler equations on GPUs. A mesh curvature approach is presented for the proper resolution of the domain boundary. This approach is based on the linear elasticity equations and enables a boundary approximation with arbitrary, high order. In order to demonstrate the performance of the boundary curvature a massively parallel solver on graphics processors is implemented and utilized for the solution of the Euler equations of gas-dynamics.

List of symbols

α	Angle of attack
$D_{r,s,t}$	Differentiation matrices
$\mathcal{E}$	Young's modulus
E	Total energy
ϵ	Strain tensor
η	Artificial viscosity coefficient
F	Flux tensor
F^*	Numerical flux function

Communicated by: Gabriel Wittum.

M. Siebenborn · V. Schulz (✉)
 Universität Trier, Universitätsring 15, 54296 Trier, Germany
 e-mail: Volker.Schulz@uni-trier.de

M. Siebenborn
 e-mail: Siebenborn@uni-trier.de

S. Schmidt
 Department of Aeronautics, Imperial College London,
 South Kensington Campus, London SW7 2AZ, UK
 e-mail: s.schmidt@imperial.ac.uk

g	Deformation on curved boundary
$\hat{f}$	Modal expansion coefficients
f	Nodal values
h	Linear elasticity solution
$\mathcal{I}_{cub}$	Cubature interpolation matrix
$\mathcal{I}_g$	Gauss point interpolation matrix
J_i^{cub}	Jacobian at i th cubature node
J_i^g	Jacobian at i th Gauss node
κ	Coefficient for smoothness indicator
l_j	Lagrange polynomials
λ, μ	Lamé parameter
M	Mass matrix
M_∞	Free stream Mach number
$M_{\partial\Omega}$	Face mass matrix
N_g	Number of Gauss nodes
N_{cub}	Number of cubature nodes
N_p	Number of collocation nodes
n	Number of components of U
ν	Poisson's ratio
Ω	Domain of interest
Ω_k	k th element
Ω_c	Region of curved elements
p	Degree of basis polynomials
p	Pressure
ψ_i	Modal basis functions
Ψ_k	Transformation functions
q_h	Vector of auxiliary variables
ρ	Density
r	Coordinate in reference element
r_i	Collocation points
r_i^{cub}	Cubature nodes in reference element
r_i^g	Gauss nodes in reference element
$S_{x,y,z}$	Stiffness matrices
σ	Stress tensor
S_k	Smoothness indicator in element k

S	NURB surface of the geometry
$\mathcal{T}$	Reference element
u	Velocity in x direction
U	Unknown function
U_h	Vector of unknown values
U_h^{cub}	Unknown values at cubature nodes
U_h^g	Unknown values at Gauss nodes
$\mathcal{V}_h$	Global ansatz space
$\mathcal{V}_h^k$	Local ansatz space
v	Velocity in y direction
V	Vandermonde matrix
w	Velocity in z direction
W_i^{cub}	Cubature weights
W_i^g	Gauss quadrature
$\mathbf{x}$	Coordinate in physical mesh

1 Introduction

The DG method is based on a discontinuous finite element spatial discretization originally introduced by Reed and Hill in the early 1970s for the neutron transport equation [1]. Later, in a series of papers [2–6], Cockburn and Shu combined the discontinuous Galerkin spatial discretization with an explicit Runge–Kutta time stepping (RKDG method) and extended the method to systems of conservation laws. This created the opportunity for highly parallel implementations, since in the RKDG method, one grid cell only needs information from the immediate neighbouring cells to march in time. Based on this, Biswas et al. investigated the potential of RKDG methods for parallelization in [7].

Another outstanding feature of DG methods is that the degree of basis functions can be chosen arbitrarily, thus leading to high order discretization. However, attention has to be paid to the representation of curved boundaries. Bassi and Rebay applied this method to the two dimensional Euler equations and worked out the importance of the boundary approximation with respect to the solution quality [8]. They pointed out that the order of the method is limited by the order of the boundary representation. Thus, it is necessary to deal with curved element discretizations, in order to overcome this issue. While this seems not to be challenging for some test cases in two dimensions it is quite difficult to approximate complex geometries arising, e.g., from industrial applications.

We present a mesh curvature procedure based on linear elasticity deformations which matches a desired boundary shape. Similar techniques are well known for the deformation of discretization meshes and avoid expensive remeshing [9]. In order to deal with discontinuities arising in the solution of hyperbolic equations, slope limiters were introduced to the RKDG method by Cockburn and Shu. Successful in the finite volume community, limiters are yet complicated for higher

order methods. This gives rise to the idea of adding artificial viscosity to the equations in order to smear out discontinuities and control the width of shocks. Persson and Peraire proposed a detector to apply artificial viscosity only in the vicinity of shocks [10]. This approach seems very promising because of its flexibility, since there is no dependency on the order of the scheme or the geometry of the discretization elements like for slope limiters.

Recently, there has been a lot of attention to discontinuous Galerkin methods because of their potential in terms of parallelization and high performance computing (HPC). Especially the nodal DG method proposed by Hesthaven and Warburton [11] was shown by Klöckner, Warburton, Bridge and Hesthaven [12] to perform very efficiently on modern graphics processors (GPUs) gaining high speedups compared to conventional codes.

In this work, we present a novel approach combining the following aspects. We implement a high order Runge–Kutta discontinuous Galerkin method for the Euler equations of gas-dynamics on GPUs. For that, we propose an approach introducing unstructured, curved-element DG discretizations into a massively parallel GPU algorithm. Furthermore, we present a mesh curvature approach, which enables high order accurate boundary representations and seamlessly fits into the DG framework. In particular, we cover the whole simulation chain from the generation of curved, body-fitted meshes up to the parallel HPC system solution. Finally, we demonstrate the performance of this algorithm on some challenging transsonic test cases, where discontinuities arise in the solutions.

This paper has the following structure. In Sect. 2, the discontinuous Galerkin method is shortly introduced and it is shown how the discrete operators are composed. It is also covered how discontinuities in the solution are handled. In Sect. 3, we describe an approach for dealing with curvature in the DG discretization. Section 4 shows the implementation on GPUs. Finally, in Sects. 5 and 6 numerical results are presented and discussed.

2 The discontinuous Galerkin method

In this paper, we study a discontinuous Galerkin method for systems of hyperbolic conservation laws of the form

$$\frac{\partial U}{\partial t} + \frac{\partial F_1(U)}{\partial x_1} + \cdots + \frac{\partial F_d(U)}{\partial x_d} = 0 \quad \text{in } (0, T] \times \Omega,$$

$$U(0, x) = U_0(x) \quad x \in \Omega$$

where $U : \mathbb{R} \times \mathbb{R}^d \rightarrow \mathbb{R}^n$, $U(t, \mathbf{x}) = (U_1(t, \mathbf{x}), \dots, U_n(t, \mathbf{x}))^T$ is the vector of conserved quantities at a point $\mathbf{x}$ in d -dimensional space and at time t . Here $\Omega \subset \mathbb{R}^d$ is the domain of interest and $[0, T]$ a time interval. The vector fields

$F_i : \mathbb{R}^n \rightarrow \mathbb{R}^n$ in this system are usually referred to as flux vectors.

For the purpose of this work we will maintain a more compact and widely used notation. Introducing the tensor $F : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times d}$, $F(U) = (F_1(U) \dots F_d(U))$ in terms of the fluxes we then obtain

$$\frac{\partial U}{\partial t} + \nabla \cdot F(U) = 0. \quad (1)$$

We follow the approaches in [13] and mostly use the notation therein. For the derivation of the discontinuous Galerkin method, we assume that the domain of interest Ω is subdivided into a finite set of K disjoint, conforming elements $\Omega = \bigcup_{k=1}^K \Omega_k$. In our approach this is a tetrahedral mesh with curved elements. The solution is then approximated using a space $\mathcal{V}_h$ of element-wise defined polynomials ψ_j up to degree p

$$\mathcal{V}_h = \bigoplus_{k=1}^K \mathcal{V}_h^k, \quad \mathcal{V}_h^k = \text{span}\{\psi_j(\Omega_k), j = 1, \dots, N_p\}.$$

Here, $N_p = \frac{(p+d)!}{d!p!}$ is the number of basis functions depending on the desired degree p and the spatial dimension d . Multiplying with a test function Φ from the same space and integration over element Ω_k leads to

$$\int_{\Omega_k} \left[\frac{\partial U}{\partial t} + \nabla \cdot F(U) \right] \Phi d\Omega = 0.$$

Integration by parts then yields the weak discontinuous Galerkin formulation

$$\begin{aligned} \int_{\Omega_k} \frac{\partial U}{\partial t} \Phi d\Omega &= - \int_{\partial\Omega_k} (F(U)^* \cdot \vec{n}) \Phi dS \\ &\quad + \int_{\Omega_k} F(U) \cdot \nabla \Phi d\Omega. \end{aligned} \quad (2)$$

Here, $\vec{n}$ denotes the outward pointing normal vector and F^* an approximate Riemann solver, which deals with the double-valued state at the element interfaces, e.g. upwinding. These approximate Riemann solvers are well known from finite volume context and a survey can be found in [14].

For the purpose of this work, we tested both a local Lax–Friedrichs and a HLLC Riemann solver but without noticing major differences.

In the following, a discrete version of Eq. (2) is derived, which can be implemented on a computer. In order to calculate the integrals occurring in (2) we have to establish a link between an arbitrary curved element Ω_k in the mesh and the reference tetrahedron $\mathcal{T} = \{-1 \leq r, s, t \leq 1, r + s + t \leq -1\}$ on which cubature/quadrature rules are known. This is done by mapping functions $\Psi_k : \Omega_k \rightarrow \mathcal{T}$ for each element as illustrated in Fig. 1 connecting the physical coordinates $\mathbf{x} = (x, y, z)$ to the computational ones $\mathbf{r} = (r, s, t)$. Moreover, the partial derivatives of Ψ_k have to be known since integration for any functions $f, g : \Omega_k \rightarrow \mathbb{R}$ in the physical space is evaluated as

$$\int_{\Omega_k} f g d\Omega = \int_{\mathcal{T}=\Psi_k(\Omega_k)} f(\Psi_k) g(\Psi_k) |\det(D\Psi_k)| d\mathcal{T}$$

with the convention

$$D\Psi = \begin{pmatrix} r_x & r_y & r_z \\ s_x & s_y & s_z \\ t_x & t_y & t_z \end{pmatrix} \quad \text{and} \quad J = |\det(D\Psi)|. \quad (3)$$

Hence, it is sufficient to derive the local operators for the reference element and transform them into the physical space. Then, we introduce an orthonormal, hierarchical set of modal basis functions $\{\psi_i, i = 1, \dots, N_p\}$ on the reference element $\mathcal{T}$. An arbitrary function f on $\mathcal{T}$ is then interpolated on a given set of collocation points $\{\mathbf{r}_i, i = 1, \dots, N_p\}$ (Fig. 2a) as

$$f(\mathbf{r}) = \sum_{j=0}^{N_p} \hat{f}_j \psi_j(\mathbf{r}), \quad \forall i = 1, \dots, N_p,$$

with modal expansion coefficients $\hat{\mathbf{f}} = (\hat{f}_1, \dots, \hat{f}_{N_p})^T$. The interpolation can be reformulated in terms of a multivariate Lagrange polynomial basis

$$f(\mathbf{r}) = \sum_{j=0}^{N_p} f(\mathbf{r}_j) l_j(\mathbf{r}), \quad \forall i = 1, \dots, N_p$$

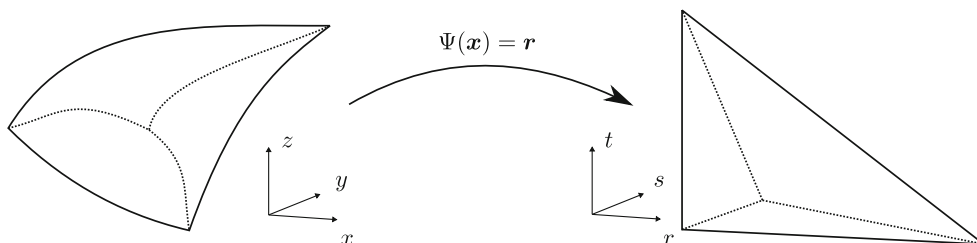


Fig. 1 Mapping from physical to reference element

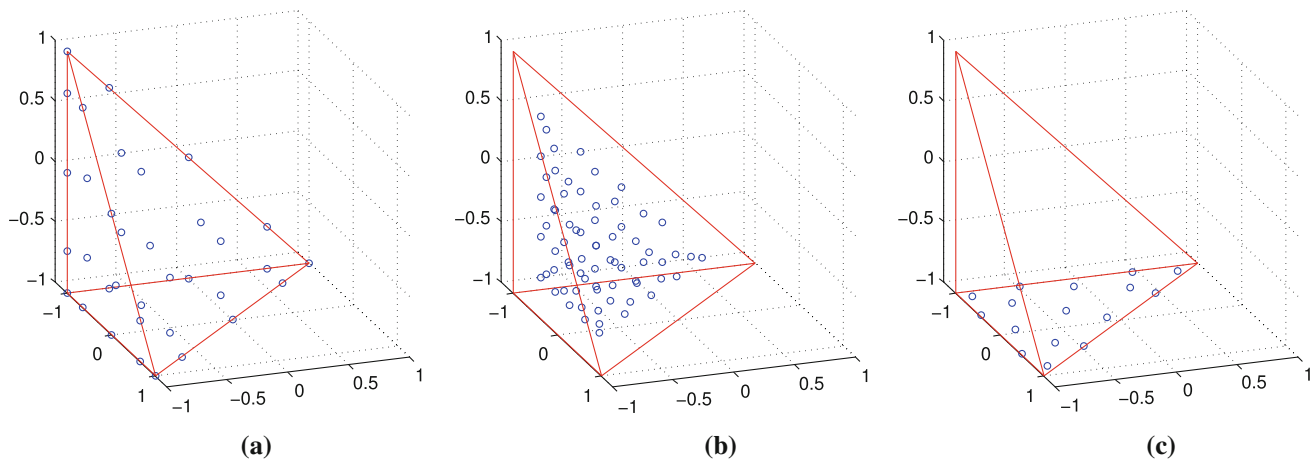


Fig. 2 Collocation and quadrature points. **a** Collocation points $\mathbf{r}_i$. **b** Volume cubature points $\mathbf{r}_i^{\text{cub}}$. **c** Surface quadrature points $\mathbf{r}_i^g$

with nodal values $\mathbf{f} = (f(\mathbf{r}_1), \dots, f(\mathbf{r}_{N_p}))^T$. These grid points are chosen according to [15] to ensure good interpolation properties. Introducing the Vandermonde matrix $V_{ij} = \psi_j(\mathbf{r}_i)$ we obtain that these two formulations are linked as

$$\mathbf{f} = V \hat{\mathbf{f}}.$$

Since there are as many basis polynomials as collocation points and depending on the interpolation quality V is nonsingular. Now, we are prepared to define the local operators for the integration in (2). For that purpose, we distinguish three sets of nodes: the collocation points $\mathbf{r}_i$ as introduced above and two sets of nodes used for integration. These are on the one hand cubature points $\{\mathbf{r}_i^{\text{cub}}, i = 1, \dots, N_{\text{cub}}\}$ (Fig. 2b) for volume integrals [16] and on the other Gauss quadrature points $\{\mathbf{r}_i^g, i = 1, \dots, N_g\}$ (Fig. 2c) for surface integrals [17]. For the derivation of the discrete operators, we closely follow [13]. First, in order to interpolate the solution given at the nodal points $\mathbf{r}_i$ to quadrature points $\mathbf{r}_i^{\text{cub}}$ and $\mathbf{r}_i^g$ we need the following interpolation matrices

$$\mathcal{I}_{\text{cub}} = V_{\text{cub}} V^{-1} \in \mathbb{R}^{N_{\text{cub}} \times N_p}, \quad \mathcal{I}_g = V_g V^{-1} \in \mathbb{R}^{N_g \times N_p}$$

where $(V_{\text{cub}})_{ij} = \psi_j(\mathbf{r}_i^{\text{cub}})$ and $(V_g)_{ij} = \psi_j(\mathbf{r}_i^g)$. Multiplying the vector of nodal unknown values with these matrices first transforms them to modal expansion coefficients and then evaluates the modal basis functions at the cubature points.

For the construction of the stiffness matrices we need to evaluate derivatives like $\frac{\partial l_i}{\partial \mathbf{r}}$ at the cubature points. Here we introduce the operator

$$D_r = V_r V^{-1} \in \mathbb{R}^{N_{\text{cub}} \times N_p}$$

where $(V_r)_{ij} = \frac{\partial \psi_j(\mathbf{r})}{\partial \mathbf{r}} \Big|_{\mathbf{r}=\mathbf{r}_i^{\text{cub}}}$. The s and t derivatives are analogous.

With this we are now prepared to assemble the local stiffness matrix for one specific element leaving out the element number k for clarity

$$S_x = \left(D_r^T \cdot \text{diag}(r_{x,i}) + D_s^T \cdot \text{diag}(s_{x,i}) + D_t^T \cdot \text{diag}(t_{x,i}) \right) \cdot \text{diag}(J_i W_i^{\text{cub}}) \in \mathbb{R}^{N_p \times N_{\text{cub}}} \quad (4)$$

where W_i^{cub} are cubature weights. S_y and S_z are obtained in the same way. The local mass matrices are then given by

$$M = \mathcal{I}_{\text{cub}}^T \cdot \text{diag}(J_i^{\text{cub}} W_i^{\text{cub}}) \cdot \mathcal{I}_{\text{cub}} \in \mathbb{R}^{N_p \times N_p} \quad (5)$$

Finally, we obtain the local face mass matrices

$$M_{\partial\Omega} = \mathcal{I}_g^T \cdot \text{diag}(J_i^g W_i^g) \in \mathbb{R}^{N_p \times N_g} \quad (6)$$

where W_i^g are the weights of a 2d Gauss integration rule for triangles. Plugging these operators into Eq. (2) yields the semi-discrete system

$$\frac{\partial U_h}{\partial t} = M_k^{-1} \sum_{m=1}^3 S_{k,x_m} F_m(U_h^{\text{cub}}) - M_k^{-1} M_{\partial\Omega_k} \left(F(U_h^g)^* \cdot \vec{n} \right). \quad (7)$$

In the equation above, $U_h^{\text{cub}} = \mathcal{I}_{\text{cub}} U_h$ denote the degrees of freedom associated with the cubature points used for the volume integration (Fig. 2b) and $U_h^g = \mathcal{I}_g U_h$ the ones for surface integration (Fig. 2c) respectively. Since the operators are only local and therefore represented by small matrices, a LU or Cholesky decomposition of M_k can be calculated in an initial step of the solver.

Finally, the system is discretized in time using a fourth-order accurate Runge–Kutta pseudo time stepping consisting of five stages. We use a low storage version of this scheme as described in [18] which does not require to keep the intermediate stages in memory. Furthermore, the application of an explicit time integration like this RK method does not

require to set up a global matrix system. Thus, the method above is a so called matrix-free approach which is especially in three dimensions very attractive with respect to memory requirements.

When dealing with fluid dynamics (see Sect. 5), the method described above works reliably for low Mach numbers. Yet, for higher Mach numbers, shocks might appear in the solution leading to strong non-physical oscillations. This phenomenon is well understood and can be overcome by adding a small amount of artificial viscosity to the equations as proposed by Persson and Peraire in [10].

However, in regions where the solution is smooth there is no need for stabilization. Hence, these authors compare the approximated solution of one component u_h to the solution $\tilde{u}_h$ where they drop the modes with the highest frequencies

$$u_h = \sum_{j=1}^{N_p} \hat{u}_j \psi_j, \quad \tilde{u}_h = \sum_{j=1}^{N_p-1} \hat{u}_j \psi_j$$

with the following smoothness indicator

$$S_k = \frac{\int_{\Omega_k} (u_h - \tilde{u}_h) \cdot (u_h - \tilde{u}_h) d\Omega}{\int_{\Omega_k} u_h \cdot u_h d\Omega}.$$

In smooth regions of the solution, this indicator will be close to zero whereas it increases in regions with high frequencies. The amount of artificial viscosity in element k is then determined as

$$\eta_k = \begin{cases} 0 & \text{if } s_k < s_0 - \kappa \\ \frac{\eta_0}{2} \left(1 + \sin \frac{\pi(s_k - s_0)}{2\kappa} \right) & \text{if } s_0 - \kappa \leq s_k \leq s_0 + \kappa \\ \eta_0 & \text{if } s_k > s_0 + \kappa \end{cases}$$

with $s_k = \log_{10}(S_k)$, $s_0 \sim \log_{10}\left(\frac{1}{p^{\frac{1}{3}}}\right)$ and empirically chosen parameters κ and η_0 . Thus, by applying a shock detector it can be decided where shocks arise and where to add viscosity. The modified system of equation is then given by

$$\frac{\partial U}{\partial t} + \nabla \cdot F(U) = \nabla \cdot (\eta \nabla U).$$

This can be rewritten as a system of first order equations and discretized using the same DG approach

$$\begin{aligned} \frac{\partial U}{\partial t} + \nabla \cdot F(U) - \nabla \cdot \sqrt{\eta} q &= 0 \\ q - \sqrt{\eta} \nabla U &= 0. \end{aligned}$$

Through this procedure the semi-discrete system (7) is expanded by an additional equation and the viscous fluxes yielding (c.f. [19])

$$\begin{aligned} \frac{\partial U_h}{\partial t} &= M_k^{-1} \sum_{m=1}^3 S_{k,x_m} \left[F_m \left(U_h^{\text{cub}} \right) - \sqrt{\epsilon} q_h^{\text{cub}} \right] \\ &\quad - M_k^{-1} M_g \left[\left(F \left(U_h^g \right)^* - \left(\sqrt{\epsilon} q_h^g \right)^* \right) \cdot \vec{n} \right] \\ q_h &= M_k^{-1} \sum_{m=1}^3 \sqrt{\epsilon} S_{k,x_m} U_h^{\text{cub}} - M_k^{-1} M_g \left[\left(\sqrt{\epsilon} U_h^g \right)^* \cdot \vec{n} \right]. \end{aligned}$$

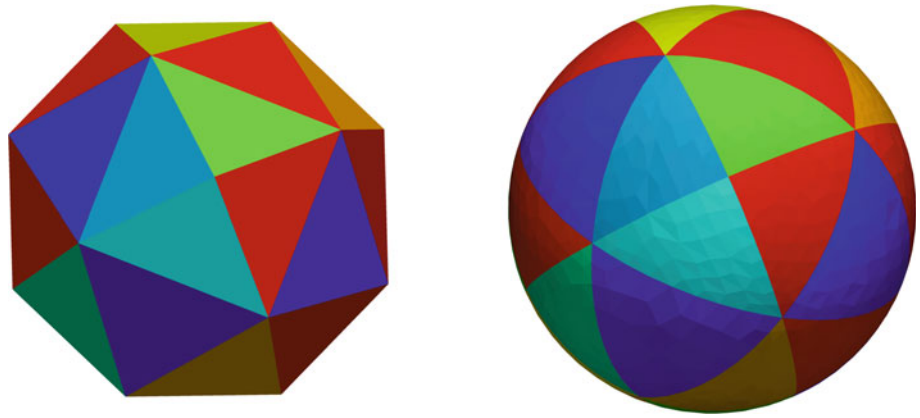
3 Mesh generation for DG

As pointed out in the introduction, a remarkable feature of the DG method is the freedom to choose the degree of basis functions. However, because of that the mesh has to be dealt with caution. It is tempting to reuse the meshes from the finite volume community. Yet, this leads to major problems, since the discretization with straight-sided elements leads to kinks at the boundary walls, which is problematic whenever the fluid is interacting with curved geometries. In this situation the numerical solution might contain small non-physical shocks in each element at the boundary. In principle, this could be overcome by using a very fine discretization. However, this is conceptually in contrast to higher order discretization methods. Thus, it is necessary to introduce curved elements into the discontinuous Galerkin discretization in order to enable a higher order boundary representation.

An isoparametric mapping is applied for the realization of the element curvature. Assuming that the deformation of the elements can be represented in the same Lagrange polynomial basis as used for the DG scheme, we only need to know the displacement of the collocation points with respect to the ones in the straight sided element. Thus, it is not enough to have a functional description of the curved boundary. Additionally, the locations of the collocation points have to be corrected such that they do not fall out of the element. Moreover, this displacement procedure should work smoothly, in order not to perturb the integration.

In two dimensions this process is simple. Here, only the elements sharing a face with the boundary wall are affected. By ensuring that the collocation points at the two other faces remain on their initial position, the neighboring elements remain untouched, even if they share one vertex with the boundary. Obviously, this does not hold for the three dimensional case. By deforming a face on the boundary the other three faces in general have to be curved as well. This forces a whole layer of cells around the wall to be curved.

In the following, we focus on how to transport the curvature information of the boundary into the mesh. As mentioned before, in three dimensions a lot of cells are affected. Hence, we suggest instead of tracking these cells to put a bounding box around the geometry marking an area in which cells get curved. We follow the ideas in [9] in order to model the

Fig. 3 Deformation to a sphere

discretization mesh as embedded in some flexible material and solve the linear elasticity equations with a finite element method. By this, the curvature information of the boundary is transported into the mesh and can be retrieved at arbitrary points. In order to keep this process as cheap as possible, we extract the cells which should become curved from the CFD-mesh and solve the linear elasticity equation on the resulting sub-mesh. The governing equations of the linear elasticity are given by

$$\nabla \cdot \sigma = f \quad \text{on } \Omega_c.$$

Here f denotes body forces acting on the solid, which will not be present in our case. $\Omega_c \subset \Omega$ is the region of the DG mesh containing the cells to be curved. Finally, σ denotes the stress tensor and is defined by the strain tensor ϵ as

$$\sigma = \lambda \text{Tr}(\epsilon)I + 2\mu\epsilon, \quad \epsilon = \frac{1}{2}(\nabla h + \nabla h^T).$$

In the equations above, λ and μ are the Lamé parameters, which can be expressed in terms of the Young's modulus $\mathcal{E}$ and Poisson's ratio ν

$$\lambda = \frac{\nu\mathcal{E}}{(1+\nu)(1-2\nu)}, \quad \mu = \frac{\mathcal{E}}{2(1+\nu)}.$$

The vector-valued function $h : \Omega_c \rightarrow \mathbb{R}^3$ describes the displacement which the material would perform under the prescribed forces. In our case, it is not important whether the chosen coefficients represent a physical material. Nevertheless, it is rather crucial to know how they act on the solution. Young's modulus can be seen as a measure for the stiffness and Poisson's ratio describes how much a material expands in two coordinate directions when compressed in the third. Since $f = 0$ in our case the solution does not depend on the stiffness parameter $\mathcal{E}$. Thus, the mesh deformation is controlled by ν only. This parameter is then chosen depending on the situation. If we want to compute the flow past a sphere, like illustrated in Fig. 3, we apply a rubber-like material ($\nu \approx 0.5$) in order to obtain a smooth deformation, which propagates in each coordinate direction.

In the NACA0012 case, which is a 3d staggered version of the 2d test case, we choose ν close to 0 in order to avoid deformation into the third dimension. This system is then completed by the boundary conditions

$$h = g \quad \text{on } \Gamma_{D1}, \quad (8)$$

$$h = 0 \quad \text{on } \Gamma_{D2}, \quad (9)$$

$$\frac{\partial h}{\partial \vec{n}} = 0 \quad \text{on } \Gamma_N. \quad (10)$$

We choose the boundaries such that Γ_{D1} is the objects surface. Γ_{D2} is the bounding box defining the sub-mesh for the elasticity equation. Finally, Γ_N is used to model symmetry walls, where collocation points are allowed to slide but may not leave the plane. One great advantage of this approach is the variable order of boundary representation. Since, we are free to choose the degree of the finite element basis functions, we can fit the boundary representation to the order of the DG scheme. Moreover, in most situations this approach avoids singularities due to the deformation. This means, if the gap between the straight-sided mesh and the curved boundary is not too large, we can expect a feasible deformation of the mesh without overlappings or zero-volume cells (Figs. 4, 5).

However, this approach requires a parameterized representation of the surface, since the displacements at the boundary have to be evaluated at arbitrary coordinates. While this seems not to be a problem for many geometries existing as CAD models, we are also dealing with geometries only available as a set of vertices. Yet, there is a lot of literature dealing with the problem how to find a NURB surface representation to a given set of points.

For a parameterized surface it must be decided how the straight-sided mesh must be mapped to fit the curved geometry. For this purpose a closest point problem has to be solved

$$\begin{aligned} \min_{(a,b)} \quad & \frac{1}{2} \|\mathbf{x} - \mathcal{S}(a, b)\|^2 \\ \text{s.t.} \quad & 0 \leq a, b \leq 1 \end{aligned} \quad (11)$$

where $\mathcal{S}(\cdot, \cdot)$ denotes a NURB surface and $\mathbf{x}$ is a point on the straight-sided boundary. We solve this problem by the Gauss-Newton algorithm. Let (a^*, b^*) be the optimal solution, then $g(\mathbf{x}) = \mathcal{S}(a^*, b^*) - \mathbf{x}$ is plugged into the boundary condition on Γ_{D1} .

With these boundary conditions we are now able to apply a high order finite element solver for the linear elasticity equation. For this purpose we use the GETFEM finite element toolbox. Then, we store the solution obtained together with the CFD-mesh and use it later in the discontinuous Galerkin solver in order to retrieve mesh displacement information at arbitrary locations.

4 Application on GPUs

Reviewing the DG method described above, it turns out that there is a lot of parallelism in the executions. On the coarsest level one can proceed as in other discretization techniques like finite volume methods and partition the mesh. Due to the nature of DG this does not lead to much data transfer between the compute nodes since only values at the element faces have to be communicated. The same holds on a second level of parallelism. As seen in the derivation of the DG method in Sect. 2, there is only a loose coupling between elements. Most operations can be performed independently. Just the evaluation of the surface integral requires the nodal values on the face of the neighbouring element to be known.

Furthermore, we can distinguish a third level of parallelism, the most interesting one. Both on the volume and on the surface quadrature nodes, the nonlinear flux function F has to be computed, which is a very expensive operation. Not only the high order basis functions but also the rational Jacobian of the non-affine mapping force us to apply a very high order quadrature rule leading to a huge number of quadrature points. However, these operations are also independent per node.

These features suggest the application of graphics processing units, i.e. GPUs, to the DG method, since the design considerations above perfectly fit into the CUDA hardware model. Originally designed to render many geometrical primitives or pixels in parallel the Nvidia Fermi architecture nowadays is based on a set of 16 so called streaming multiprocessors (SM). Each of them features 32 cores which leads to a total number of 512 CUDA cores enabling hundreds of floating point operations at a time.

From the programming point of view, CUDA organizes threads in a hierarchical structure. On the lowest level there are the CUDA threads each having its own registers for temporary variables. These threads are then gathered in thread blocks and identified through a three dimensional index. Each of these blocks comes with up to 64 kb shared memory in which data can be collected and distributed between the

threads. One major advantage of this concept is that shared memory is on chip. Thus, data which is used multiple times or has to be shared between threads has to be requested from the global RAM of the GPU only once. Finally, on the top level blocks are organized in a grid structure and again identified through three dimensional indices.

For further details we refer to the CUDA documentation [20]. Yet, one crucial aspect should be addressed. In most cases the performance gain of a massively parallel GPU algorithm strongly depends on memory fetching strategies. Generally, whenever the threads of a block $\{t_0, \dots, t_n\}$ fetch data from an array $A = \{a_0, \dots, a_N\}$ out of the global RAM of the GPU, these accesses should be organized in a block-wise fashion, where the blocksize is a multiple of 16. The threads $t_0, \dots, t_n$ should access successive data elements - each thread one element—where the first index is a multiple of 16. Thus, memory accesses into the i th block should be organized as

$$t_{\tau(j)} \rightsquigarrow a_{(16i+j)}, \quad j \in \{0, \dots, n\}$$

where τ is a permutation on $\{0, \dots, n\}$. In this case, multiple memory accesses can be performed at once leading to so called coalesced accesses. Whereas, when a thread block requests data which is scattered throughout the memory this ends up in serialization of the accesses.

This issue is crucial particularly for older GPU hardware where finite volume methods (FVM) have been implemented for fluid dynamics. Since the FVM can be seen as a special case of the DG method for $p = 0$, there is only one unknown per cell. All neighbouring variables are involved in the update of the value of this unknown, which leads to a high ratio of scattered data accesses in case of unstructured meshes. While there are only a few publications dealing with FVM for the Euler equations on unstructured meshes using GPUs (e.g. [21]) the DG method overcomes this issue and makes it attractive for GPU implementations. Due to the higher order of the method, there are plenty of values per cell which can be organized successively in memory allowing high speedups even on unstructured meshes.

This hardware and programming layout can now be identified with the basic steps of the DG method. As previously mentioned, operations on one element are mostly independent of the executions in other elements. Thus, it is obvious to assign one CUDA block to each DG element. Then, the arithmetically expensive nonlinear flux evaluations can be performed in parallel. Moreover, the subsequent matrix vector products can also be handled very efficiently within each block since we are dealing with small, dense matrices. Finally, the CUDA execution handler decides how many thread blocks can be executed in parallel depending on the order of the DG method.

We have to reorganize the unknown values in order to meet the memory pattern of the GPU. The length of each field

should be a multiple of 16 which is achieved by so-called zero padding as demonstrated in (12). Here, U_h^k denotes the unknown values inside element k . As illustrated in chapter 2 we are dealing with a vector of n unknowns interpolated at N_p collocation points. Thus, the padding length is determined such that the block length is enlarged from N_p to $b_p = \lceil \frac{N_p+15}{16} \rceil$ and we ensure that the start address of each field is a multiple of 16

$$U_h^k = \begin{bmatrix} U_0^1, U_1^1, \dots, U_{N_p-1}^1, 0, \dots, 0, \\ U_0^2, U_1^2, \dots, U_{N_p-1}^2, 0, \dots, 0, \\ \vdots \quad \vdots \quad \quad \quad \vdots \quad \vdots \quad \quad \quad \vdots \\ U_0^n, U_1^n, \dots, U_{N_p-1}^n, 0, \dots, 0 \end{bmatrix}. \quad (12)$$

padding

The same works for the unknowns at the quadrature points. In contrast, the values at the cubature points are only used once and do not need to get shared with other elements. Thus, we do not need to store them in the global RAM.

In the following, we will consider the element specific volume integral in Eq. (2) as an example. For the integrations inside each element, we also need to fetch the element specific local operators from the global GPU RAM. Thus, they are stored in a similar way like the vector of unknowns. For example the S_x operator in Eq. (4) is stored as

$$S_x = \begin{bmatrix} s_{(0,0)}^x, & s_{(1,0)}^x, & \dots, & s_{(N_p-1,0)}^x, & 0, \dots, & 0, \\ s_{(0,1)}^x, & s_{(1,1)}^x, & \dots, & s_{(N_p-1,1)}^x, & 0, \dots, & 0, \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ s_{(0,N_{\text{cub}}-1)}^x, & s_{(1,N_{\text{cub}}-1)}^x, & \dots, & s_{(N_p-1,N_{\text{cub}}-1)}^x, & 0, \dots, & 0 \end{bmatrix}. \quad (13)$$

padding

In order to store the interpolation matrix $\mathcal{I}_{\text{cub}}$ the block length has to be chosen as $b_{\text{cub}} = \lceil \frac{N_{\text{cub}}+15}{16} \rceil$ leading to

$$\mathcal{I}_{\text{cub}} = \begin{bmatrix} \iota_{(0,0)}, & \iota_{(1,0)}, & \dots, & \iota_{(N_{\text{cub}}-1,0)}, & 0, \dots, & 0, \\ \iota_{(0,1)}, & \iota_{(1,1)}, & \dots, & \iota_{(N_{\text{cub}}-1,1)}, & 0, \dots, & 0, \\ \vdots & \vdots & & \vdots & \vdots & \vdots \\ \iota_{(0,N_p-1)}, & \iota_{(1,N_p-1)}, & \dots, & \iota_{(N_{\text{cub}}-1,N_p-1)}, & 0, \dots, & 0 \end{bmatrix}. \quad (14)$$

padding

Note that these operators are stored in column-major form, which is important for the matrix-vector product.

With this data structures in global GPU RAM we implement the CUDA grid to have K thread blocks, which will be associated with the DG elements. Each of these blocks should contain N_{cub} threads $\{t_0, \dots, t_{N_{\text{cub}}-1}\}$ and a matrix $U[n][N_{\text{cub}}]$ in shared memory to cache the unknown values. At this point, we can assume that the matrix U is large enough

to store both U_h and U_{cub} since $N_{\text{cub}} > N_p$. This holds, because for a proper integration of the nonlinear flux function there have to be more cubature points than collocation points. The initial step is to cache the nodal values U_h into the shared matrix. For that purpose the first N_p threads load and store the n fields of U_h into U . Then the product $\mathcal{I}_{\text{cub}} U_h^k$ is evaluated column-wise. The threads $t_0, \dots, t_{N_{\text{cub}}-1}$ load one column of $\mathcal{I}_{\text{cub}}$, each thread one value. Since $\mathcal{I}_{\text{cub}}$ is stored in column-major form, these loads are coalescent. Then the n values in U corresponding to this column are broadcasted to all threads and the multiplication is performed. The broadcast operation is very efficient, since the matrix U was fetched to the shared memory which is on chip in contrast to the global GPU RAM. This is done successively for all columns and each thread is tracking the sum over all columns in its registers. Finally, by this procedure, we obtain the nodal values interpolated to the cubature points U_h^{cub} , which are again stored in U . Through this procedure the dot products between the N_{cub} rows of $\mathcal{I}_{\text{cub}}$ and the fields of U are performed in parallel each by one CUDA thread.

In order to calculate the volume integral in (2) which is approximated in the discrete system (7) by $\sum_{m=1}^3 S_{k,x_m} F(U_h^{\text{cub}})$ the threads $t_0, \dots, t_{N_{\text{cub}}-1}$ evaluate the flux function F on U in parallel. Finally, the multiplication with the three stiffness matrices $S_{x_1}, S_{x_2}, S_{x_3}$ is treated as described above. However, this matrix vector product can not be handled as one thread per output value, since S_{x_i} has more columns than rows in general. Thus, the threads are distributed over several matrix columns, which are then handled in parallel to maintain the occupancy of the streaming processors high.

For this implementation, we were inspired by the MIDG code [22]. MIDG is a very lightweight discontinuous Galerkin solver for Maxwell's equations designed to run on multiple GPUs. A mesh partitioning with ParMetis is involved which minimizes the cuts between processes and thus reduces communication. The time discretization is realized by a low storage Runge–Kutta scheme as described in Sect. 2. For the spatial discretization, the nodal DG scheme presented in [13] is applied. We mostly adopted the parallel framework of this code including the mesh partitioning and the MPI communications. However, the quadrature free approach therein does not support curved elements and leads to errors when dealing with nonlinear PDEs like the Euler equations (cf. Sect. 5). We thus applied a computationally more expensive, quadrature based scheme as introduced in Sect. 2.

An overview of our algorithm is given in Fig. 6. We work on a compute cluster—shared or distributed memory—on which each node is equipped with one GPU. Here, the executions on compute node j are described and it can be seen that after the interpolation to the quadrature nodes on the element faces the communication with other processes takes

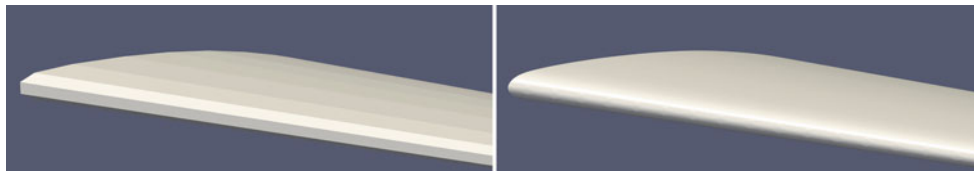
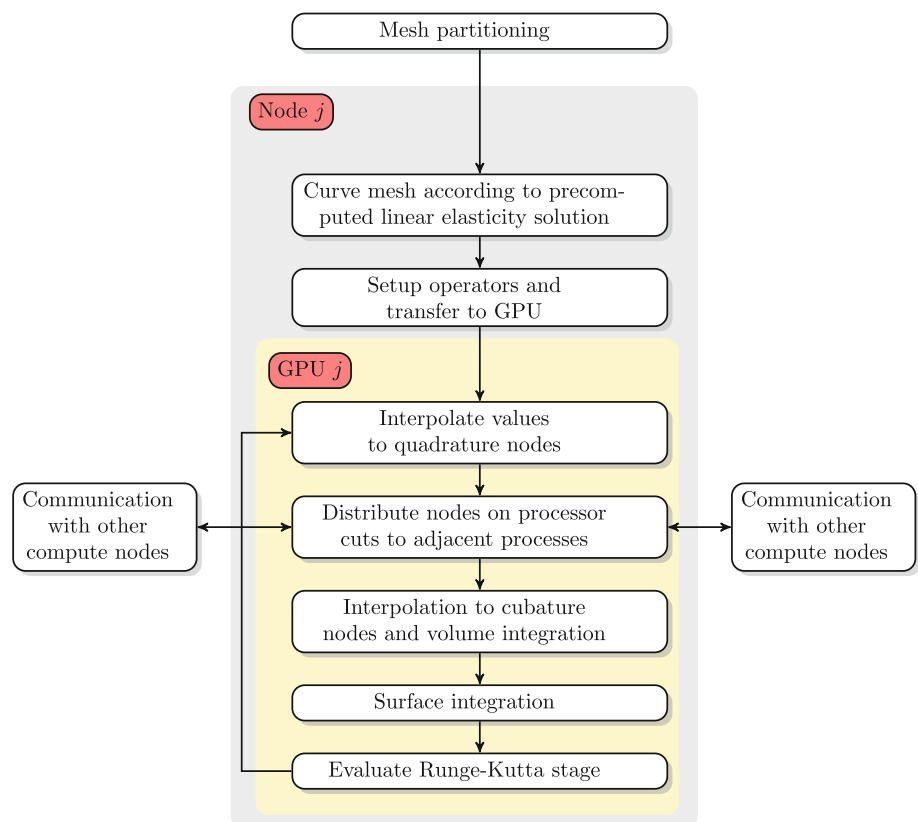


Fig. 4 Leading edge of ONERA M6 airfoil surface—initial and curved geometry



Fig. 5 Tip of NACA0012 profile—initial and curved grid

Fig. 6 Algorithm execution on one compute node



place. For that purpose, the values on processor cuts are downloaded from the GPU and distributed to other compute nodes. In order to save time, this is done asynchronously, while the volume integration is executed.

In our test setting, we have a shared memory system equipped with 8 Intel Xeon E5620 CPU cores and 8 Nvidia Tesla M2050 GPUs with CUDA 4.0. For measurements of the GPU accelerated code against a conventional CPU code, we

Table 1 Run time of the time stepping loop, 200 iterations on 25,956 cells, 1 GPU versus 4 CPU cores

p	CPU (4 cores) (s)	GPU (s)	GPU (float) (s)	Speedup	Speedup (float)
4	278.92	54.16	15.27	5.15	18.27
3	92.88	21.34	7.96	4.35	11.67
2	34.01	8.40	3.52	4.05	9.66

run a test on one GPU and compare it to a CPU code where the matrix vector multiplications are implemented using BLAS routines. In order to optimize the code for this specific hardware we apply the Intel C compiler 12.0 and the Intel math kernel library. We achieve the highest performance on the CPU by implementing the multiplications with $\mathcal{I}_{\text{cub}}$, $\mathcal{I}_g$ and $M_{\partial\Omega_k}$ [cf. Eq. (7)] with BLAS.

In contrast, the evaluation of the volume contribution $\sum_{m=1}^3 S_{k,x_m} F_m(U_h^{\text{cub}})$ is merged into one matrix vector product, which is significantly faster than three separate BLAS calls. Since one of the underlying CPUs has four cores, we use an OPENMP parallelization (four parallel threads, each on one CPU core) to achieve full utilization of its capacity. For that purpose we hint the compiler to apply a parallelization of the k-loop over all elements [cf. Eq. (2)]. In the test setting of Table 1 we achieved a run time on one CPU core of 1,016.36 s for $p = 4$ elements. Compared to 278.92 s for the parallel version on four cores this reflects the theoretical speedup on this hardware architecture.

Furthermore, Table 1 shows the run times for 200 Runge–Kutta iterations on a mesh with 25,956 cells for the CPU and GPU code. Here, the most interesting polynomial degrees with respect to CFD are inspected. For the GPU run times we distinguish a single precision (float) and a double precision implementation. The last two columns of Table 1 show the gained speedup compared to the CPU code. We observe that the speedup increases with the polynomial order. This stems from the higher number of quadrature points in one element leading to a better utilization of the available CUDA threads. Overall, on the most interesting level for this paper of $p = 4$ elements, speedup factors of 5 up to 18 for single and double precision are achieved.

5 Numerical results

As a special case of Eq. (1) we consider the Euler equations of gas-dynamics in three spatial dimensions. These equations describe the motion of an inviscid fluid without heat conduction

$$\begin{bmatrix} \rho \\ \rho u \\ \rho v \\ \rho w \\ \rho E \end{bmatrix}_t + \begin{bmatrix} \rho u \\ \rho u^2 + p \\ \rho uv \\ \rho uw \\ u(\rho E + p) \end{bmatrix}_x + \begin{bmatrix} \rho v \\ \rho uv \\ \rho v^2 + p \\ \rho vw \\ v(\rho E + p) \end{bmatrix}_y$$

$$+ \begin{bmatrix} \rho w \\ \rho uw \\ \rho vw \\ \rho w^2 + p \\ w(\rho E + p) \end{bmatrix}_z = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Here, the unknown function reads as

$$U : \mathbb{R} \times \mathbb{R}^3 \rightarrow \mathbb{R}^5, \quad U(t, \mathbf{x}) = (\rho(t, \mathbf{x}), \rho u(t, \mathbf{x}), \rho v(t, \mathbf{x}), \rho w(t, \mathbf{x}), \rho E(t, \mathbf{x}))^T$$

where ρ denotes the density, u, v, w the fluid velocities in the three space directions and E the total energy. Finally, the system is completed by the perfect gas law for the pressure

$$p = (\gamma - 1) \left(\rho E - \frac{u^2 + v^2 + w^2}{2\rho} \right)$$

where γ is the adiabatic index of the fluid [23].

Our first test case is a subsonic flow past a sphere at $M_\infty = 0.38$ which was examined together with the discontinuous Galerkin method in [8]. This test case is very attractive, since the surface curvature is straightforward. Nevertheless, we apply our curvature approach to obtain a smooth volume mesh. Let r denote the radius and $\mathbf{x}_0$ the centroid of the sphere. Then the boundary conditions in Eq. (8) for the mesh deformation are given by

$$g(\mathbf{x}) = \mathbf{x}_0 + \frac{r}{\|\mathbf{x} - \mathbf{x}_0\|} \mathbf{x} - \mathbf{x}.$$

As shown in Fig. 3, we start with a straight sided grid and solve the linear elasticity equation on the surrounding volume grid with $p = 4$ basis functions. Thus, we obtain a smoothly curved volume discretization, which we then use as input for the DG Euler solver also based on $p = 4$ basis functions. We note that best results are obtained when the order of the linear elasticity solution matches the order of the DG solver. In this case the deformation lies within the range of the DG basis functions and can be resolved properly.

Figure 7 shows the effect of curved boundaries on the solution quality. On the left hand side the solution is disturbed due to straight sided elements in the discretization mesh. This leads to small yet problematic kinks between the elements on the surface and results in a non-physical solution. In contrast, the computations on the right hand side were performed on a curved grid, which is in addition coarser. In this case the linear elasticity deformation was calculated using polynomials of order four.

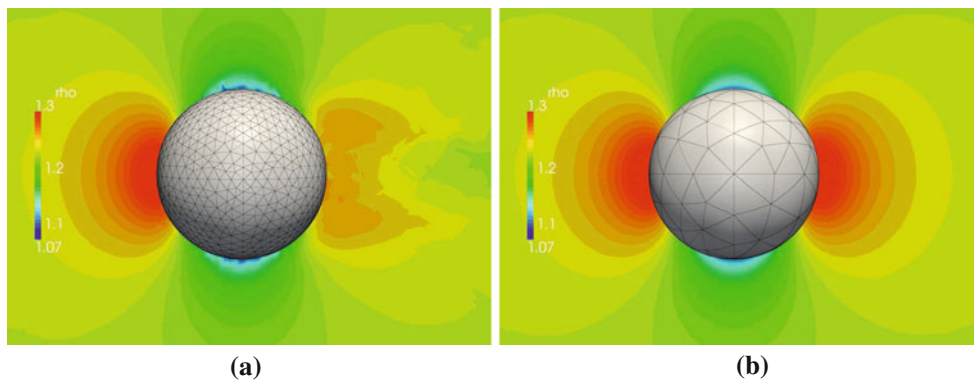


Fig. 7 Density distribution of a subsonic flow past a sphere with $p = 4$ basis functions. **a** Disturbed solution on straight sided elements. **b** Isoparametric curved elements

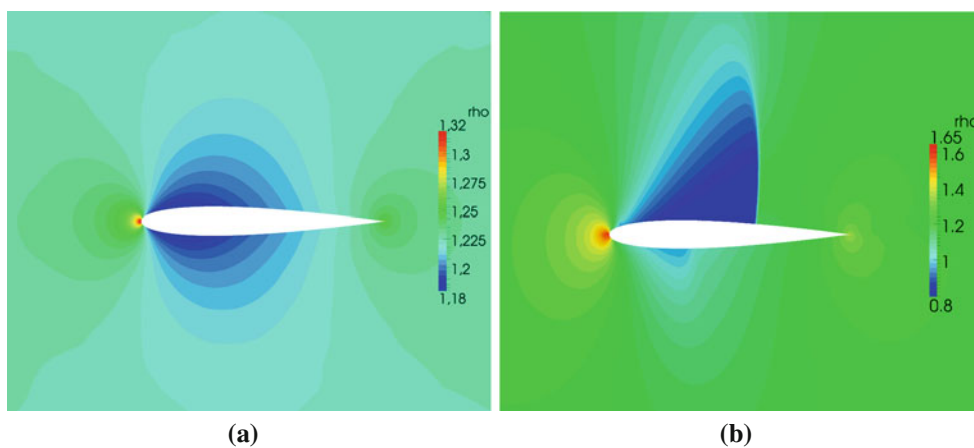


Fig. 8 Flow past NACA0012 airfoil with $p = 4$ elements. **a** Subsonic flow, $M_\infty = 0.4$, $\alpha = 0^\circ$. **b** Transonic flow, $M_\infty = 0.8$, $\alpha = 1.25^\circ$

The next test case is the NACA0012 symmetric airfoil. Although the geometry is only two dimensional, we stretch the profile into the third coordinate direction in order to apply the same solver as for the other test cases. Here, the effect of the boundary deformation on the volume mesh can be visualized as Fig. 5 shows.

In the NACA0012 situation we consider two flow conditions. First, a pure subsonic flow with no angle of attack and Mach number $M_\infty = 0.4$ (Fig. 8a). Figure 8b shows a $M_\infty = 0.8$ transonic flow with angle of attack $\alpha = 1.25^\circ$. In both pictures, the density distribution is plotted. Since we are dealing with discontinuities in this situation, we have to add artificial viscosity to the system of equations in order to ensure stability of the method. For that purpose we proceed as described in Sect. 2 and apply a shock detector to select the troubled cells.

In the following we consider the Onera M6 wing. This test case is well known and examined with a variety of numerical methods. The original experiment was defined in [24] at Mach number $M_\infty = 0.8395$ and angle of attack $\alpha = 3.06^\circ$. Again, we start with a straight-sided, tetrahedral mesh and a

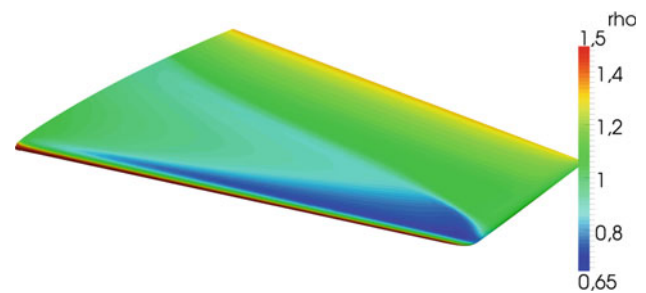


Fig. 9 Flow over Onera M6 airfoil with $p = 4$ elements

NURB representation of the ONERA M6 geometry and use the deformation approach to obtain the curved mesh (Fig. 4). Figure 9 shows the density distribution on the upper surface of the airfoil. Also in this case, artificial viscosity is applied to the system to deal with the shocks on the upper surface of the airfoil. In both the NACA0012 and ONERA M6 situation we set the parameter for the viscosity $\kappa = 4$ and $\eta_0 = 0.3$.

For this computations we utilize a so-called p -refinement as an acceleration technique. This means that we start the time stepping with low order polynomials of degree p_1 , e.g.

Table 2 Algorithm execution for the sphere—with and without p -refinement

p	Iterations	Timestep	Time consumed (s)	p	Iterations	Timestep	Time consumed
2	8,000	6e−5	105.8	2			
3	10,000	2e−5	196.5	3			
4	20,000	5e−6	650.0	4	61,000	5e−6	1982 s
	38,000		952.3		61,000		1982 s

$p_1 = 1$, iterate until the Runge–Kutta update is below a specified tolerance and then switch to a richer polynomial space of degree $p_2 = p_1 + 1$. This process is repeated until the desired order is reached. The transfer operation between the coarser and the finer polynomial space is straightforward, since we are dealing with a hierarchical set of basis functions as introduced in Sect. 2. Thus, the solution on level p_i can be directly embedded into the p_{i+1} space, where the higher order polynomials are initially weighted with zeros.

This procedure offers two advantages in terms of acceleration. Firstly, computations are much cheaper on a coarser level of p , since the number of unknown values is smaller. Furthermore, the cubature and quadrature formulas do not have to be as precise as for higher order polynomials, which leads to smaller local operators. Secondly, the timestep for the Runge–Kutta time integration scales with $\mathcal{O}(p^{-2})$, c.f. [13]. This enables a much larger timestep for smaller values of p , which reduces the number of iterations needed to reach the steady state solution.

For the computations shown in this work, we started with $p = 2$ basis functions and subsequently refined to $p = 3$ and $p = 4$. We found out that this is a convenient choice, since we are dealing with relatively coarse grids, where computations with $p = 1$ or even $p = 0$ do not lead to suitable results. Table 2 shows two different algorithm executions for the sphere test case. The underlying discretization mesh consists of 25,956 tetrahedra and 4,457 of those are curved elements. Moreover, on the finest level, each element contains 35 collocation nodes, 70 cubature nodes and $4 \cdot 16$ surface quadrature nodes. On the left hand side, the simulation was started with $p = 2$ basis functions and then subsequently refined to $p = 3$ and $p = 4$ functions. Here, the number of iterations on level 2 and 3 was chosen empirically. In contrast, on the right hand side, the simulation was run using $p = 4$ basis functions only. Both executions terminated when the infinity norm of the residual for $p = 4$ was below a tolerance of $1e-9$. For this particular case, the execution time is more than halved by the p -refinement procedure. The number of iterations on each level may look surprising, since it is multiple of 1,000. This stems from the fact that the calculation of a norm is an expensive operation in parallel. In contrast, a few extra Runge–Kutta evaluations are relatively cheap compared to the parallel reduction for the norm. Thus, we evaluate the norm of the residual only after multiples of 1,000 iterations.

6 Conclusion

In this work, we have presented a curved mesh generation approach. This approach is based on the linear elasticity deformation of an initial grid with straight sided elements until it meets the desired curved boundary. We demonstrated that the boundary approximation can be of arbitrary high order reducing the error induced into the discontinuous Galerkin discretization due to under-resolved geometries. The solution of the linear elasticity equations with a high order finite element method seems computationally very expensive at a first glance. However, this has to be done only once. We precompute the curvature and the DG solver only needs to evaluate the FEM-solution at the desired nodes.

In a second step, we showed how these curved meshes are embedded into a GPU based parallel DG solution of the Euler equations of gas-dynamics. Furthermore, the performance of this massively parallel GPU code was tested against an equivalent CPU code which is implemented using MKL-BLAS routines and parallelized with OPENMP in order to enable all cores of the underlying CPU. In this comparison we achieved speedups of up to 18. We have also chosen some challenging test cases including transonic flows leading to discontinuities which were treated with an artificial viscosity approach. Although the combination of an explicit Runge–Kutta time integration with diffusion terms leads to very small time steps we have seen that the outstanding parallel performance of the GPU overcomes this issue. And we believe that this fact justifies the additional implementation effort for the GPU code.

Acknowledgments Our research is supported by BMBF (Bundesministerium für Bildung und Forschung) within the collaborative project DGHPOPT (FKZ: 03MS632A).

References

1. Reed, W.H., Hill, T.R.: Triangular Mesh Methods for the Neutron Transport Equation. Los Alamos, Report LA-UR-73-479 (1973)
2. Cockburn, B., Shu, C.W.: The Runge–Kutta local projection P1-discontinuous Galerkin finite element method for scalar conservation laws. *RAIRO, Modélisation Mathématique et Analyse Numérique* **25**, 337–361 (1991)
3. Cockburn, B., Shu, C.W.: for conservation laws II: general framework. *Math. Comput.* **52**, 411–435 (1989)
4. Cockburn, B., Lin, S.Y., Shu, C.W.: TVB Runge–Kutta local projection discontinuous Galerkin finite element method for conser-

- vation laws III: one-dimensional systems. *J. Comput. Phys.* **84**, 90–113 (1989)
5. Cockburn, B., Hou, S., Shu, C.W.: The Runge–Kutta local projection discontinuous Galerkin finite element method for conservation laws IV: the multidimensional case. *Math. Comput.* **54**, 545–581 (1990)
 6. Cockburn, B., Shu, C.W.: The Runge–Kutta discontinuous Galerkin method for conservation laws V: multidimensional systems. *J. Comput. Phys.* **141**, 199–224 (1998)
 7. Biswas, R., Devine, K.D., Flaherty, J.E.: Parallel, adaptive finite element methods for conservation laws. *Appl. Numer. Math.* **14**, 255–283 (1994)
 8. Bassi, F., Rebay, S.: A high-order discontinuous Galerkin finite element method solution of the 2d Euler equations. *J. Comput. Phys.* **138**, 251–285 (1997)
 9. Dwight, R.P.: Robust mesh deformation using the linear elasticity equations. In: *Computational Fluid Dynamics 2006*, pp. 401–406. Springer, Berlin (2009)
 10. Persson, P.O., Peraire, J.: Sub-cell shock capturing for discontinuous Galerkin methods. In: *Proceedings of the 44th AIAA Aerospace Sciences Meeting and Exhibit*, 112 (2006)
 11. Hesthaven, J.S., Warburton, T.: Nodal high-order methods on unstructured grids: I. Time-domain solution of Maxwell’s equations. *J. Comput. Phys.* **181**, 186–221 (2002)
 12. Klockner, A., Warburton, T., Bridge, J., Hesthaven, J.S.: Nodal discontinuous Galerkin methods on graphics processors. *J. Comput. Phys.* **228**, 7863–7882 (2009)
 13. Hesthaven, J.S., Warburton, T.: *Nodal Discontinuous Galerkin Methods: Algorithms, Analysis, and Applications*. Springer, Berlin (2008)
 14. Toro, E.F.: *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction*. Springer, Berlin (2009)
 15. Warburton, T.: An explicit construction of interpolation nodes on the simplex. *J. Eng. Math.* **56**, 247–262 (2006)
 16. Grundmann, A., Möller, H.M.: Invariant integration formulas for the N-simplex by combinatorial methods. *SIAM J. Numer. Anal.* **15**, 282–290 (1978)
 17. Cools, R.: Monomial cubature rules since “Stroud”: a compilation-part 2. *J. Comput. Appl. Math.* **112**, 21–27 (1999)
 18. Carpenter, M.H., Kennedy, C.A.: Fourth-Order 2n-Storage Runge–Kutta Schemes. Nasa Report TM, 109112 (1994)
 19. Arnold, D.N., Brezzi, F., Cockburn, B., Marini, D.: Discontinuous Galerkin methods for elliptic problems. *Lect. Notes Comput. Sci. Eng.* **11**, 89–102 (2000)
 20. NVIDIA Corporation: C Programming Guide v4.0. Nvidia Corp (2011)
 21. Liebmann, M., Douglas, C., Haase, G., Horvath, Z.: Large scale simulations of the Euler equations on GPU clusters. In: Guo, Q., Guo Y., (eds.) *Proceedings of DCABES 2010*, Hongkong, pp. 50–54. IEEE Computer Society (2010)
 22. Warburton, T.: MIni DG Code. <http://www.caam.rice.edu/timwar/RMMC/MIDG.html>. (2008). (Online). Accessed 15 Nov 2010
 23. Blazek, J.: *Computational Fluid Dynamics: Principles and Applications*. Elsevier, Amsterdam (2001)
 24. Schmitt, V., Charpin, F.: Pressure Distributions on the ONERA-M6-wing at Transonic Mach Numbers. Report of the Fluid Dynamics Panel Working Group 04, AGARD AR 138 (1979)